

09 Orthogonality



9.4 Gram–Schmidt Orthogonalization

Using step-by-step projection and normalization to turn a linearly independent set into an orthonormal basis — a standard way to obtain an orthogonal matrix

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master

1

Orthogonalization: turning a linearly independent set of vectors into a pairwise-orthogonal set of unit vectors

3

Every step projects a vector onto the orthogonal complement of the basis built so far

5

$A = QR$: the resulting Q is an orthogonal matrix, and R is an upper-triangular coordinate matrix

2

Gram–Schmidt orthogonalization: achieving orthogonalization through step-by-step projection and normalization

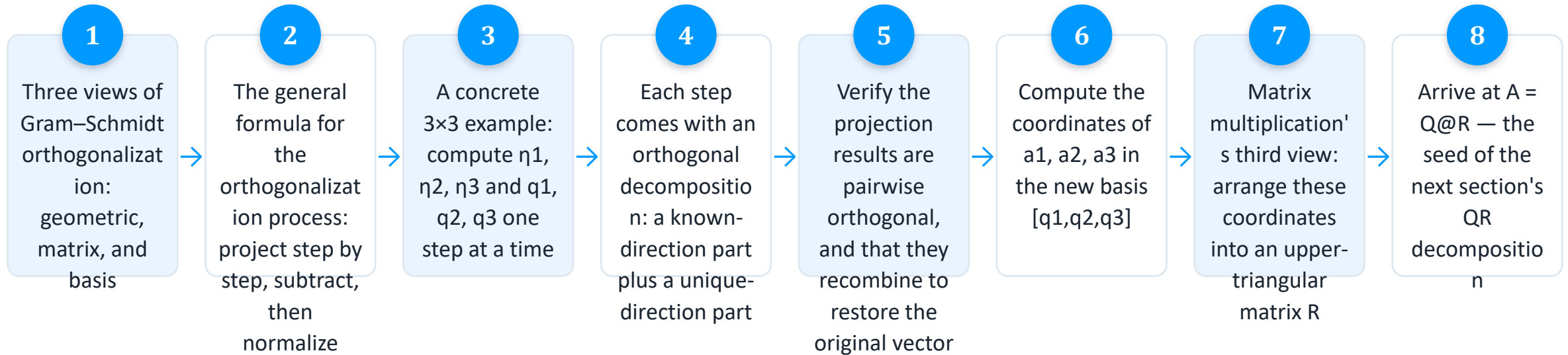
4

Every vector can be orthogonally decomposed into a "known-direction part" plus a "unique-direction part"

6

A vector's coordinates in the new basis $[q_1, q_2, q_3]$ are exactly the corresponding column of R

Starting From a Geometric View, Working Through One Complete Orthogonalization



Seven Lenses for Every Topic in This Chapter

- 1 Geometric view: the identity, rotation, reflection, permutation matrices, and their combinations are all orthogonal
- 2 Algebraic property: $\mathbf{Q}^T = \mathbf{Q}^{-1}$ — transpose equals inverse; the determinant is always +1 or -1
- 3 Gram-matrix view: the columns are pairwise orthogonal, and each column is a unit vector
- 4 Projection view: using an orthogonal matrix's columns to project — the sum of projections restores the vector
- 5 Orthogonal-complement view: every subspace has an orthogonal complement perpendicular to it everywhere
- 6 How Gram–Schmidt orthogonalization and QR decomposition build an orthonormal basis from any basis
- 7 Where orthogonal matrices show up: regression, spectral decomposition, SVD, principal component analysis

Figure 1 · The Geometric Meaning Behind Gram–Schmidt Orthogonalization

- Turns non-orthogonal $[a_1, \dots, a_p]$ into orthonormal $[q_1, \dots, q_p]$
- Yellow highlight: a_1 and q_1 are parallel

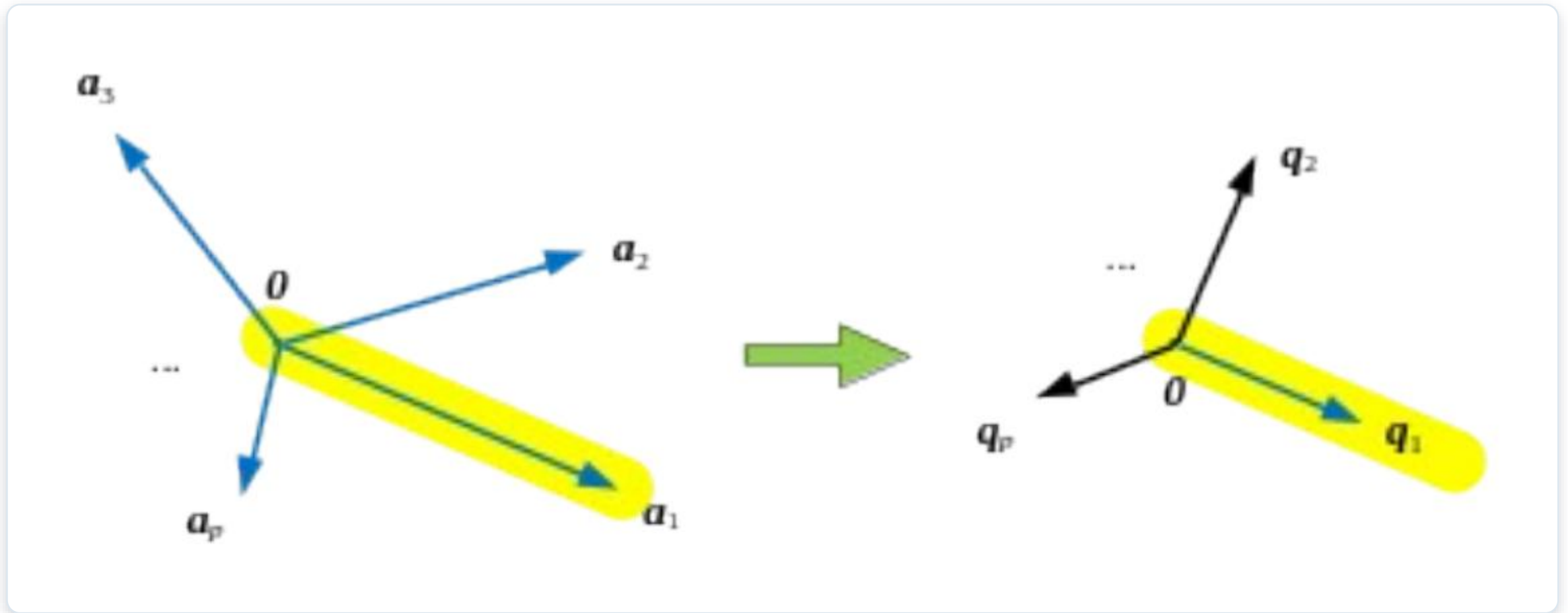
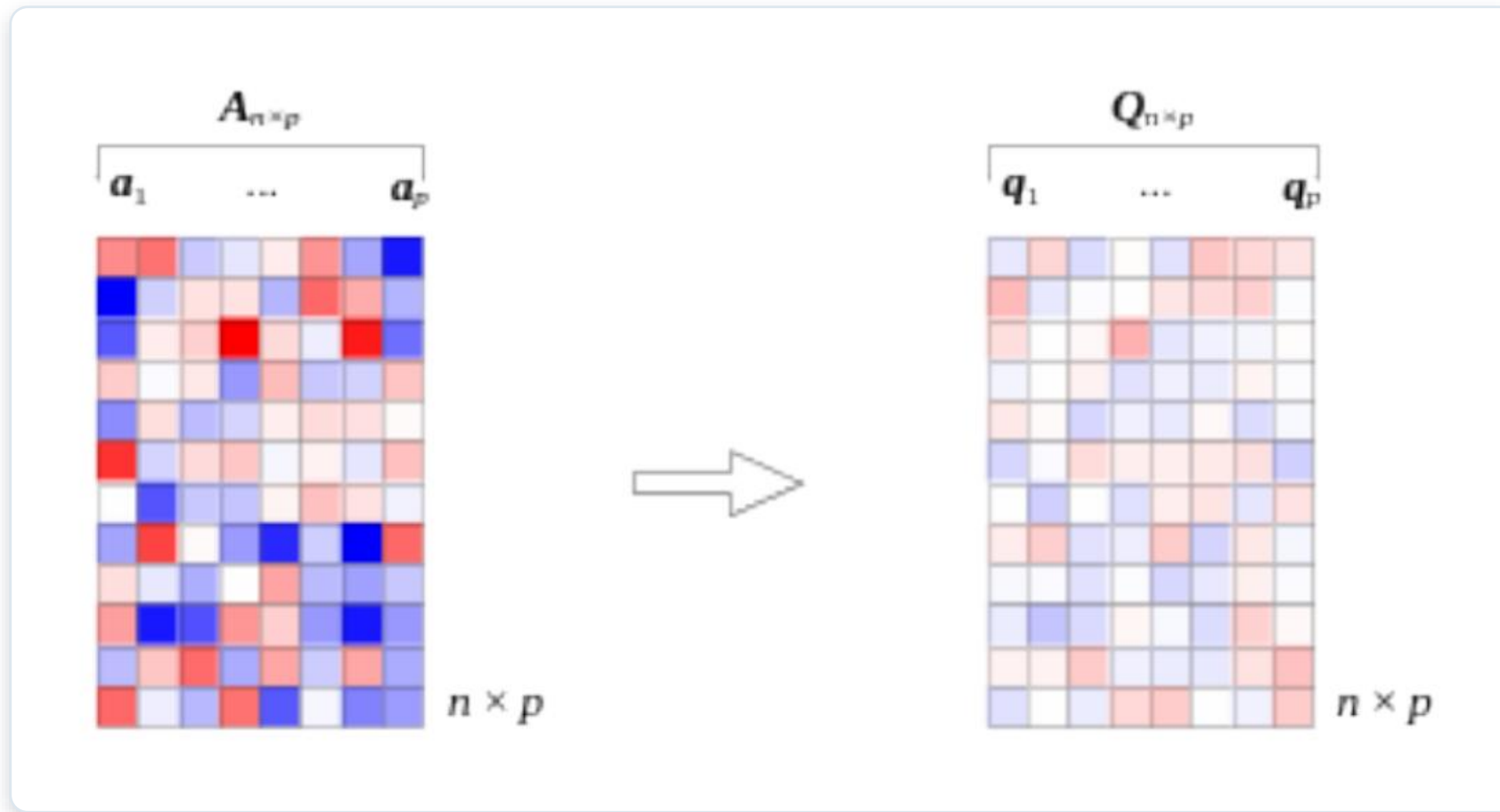


Figure 2 · A Full-Column-Rank Matrix A Becomes a Semi-Orthogonal Matrix Q

- Matrix view: full-column-rank A becomes semi-orthogonal Q
- Basis view: a non-orthogonal basis becomes orthonormal



Using vector projection, the Gram–Schmidt process is

$$\boldsymbol{\eta}_1 = \boldsymbol{a}_1 \quad \boldsymbol{\eta}_2 = \boldsymbol{a}_2 - \text{proj}_{\boldsymbol{\eta}_1}(\boldsymbol{a}_2) \quad \boldsymbol{\eta}_3 = \boldsymbol{a}_3 - \text{proj}_{\boldsymbol{\eta}_1}(\boldsymbol{a}_3) - \text{proj}_{\boldsymbol{\eta}_2}(\boldsymbol{a}_3) \quad \cdots$$

- Each step subtracts a vector's projection onto the directions already found, leaving only its "unique" component

Finally, normalize to get unit orthogonal vectors

$$\mathbf{q}_1 = \boldsymbol{\eta}_1 / \|\boldsymbol{\eta}_1\| \quad \mathbf{q}_2 = \boldsymbol{\eta}_2 / \|\boldsymbol{\eta}_2\| \quad \cdots \quad \mathbf{q}_p = \boldsymbol{\eta}_p / \|\boldsymbol{\eta}_p\|$$

- The result: each $\mathbf{q}_i$ is a unit vector, and $\mathbf{q}_i, \mathbf{q}_j$ are pairwise orthogonal
- A special feature of orthonormal basis $[\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_p]$: $\mathbf{q}_1$ is always parallel to $\mathbf{a}_1$ — this book covers other ways to build an orthonormal basis later, worth comparing

Writing the orthogonal projection in matrix form makes the process clearer

$$\text{proj}_{\eta_1}(\mathbf{a}_2) = \eta_1 @ \eta_1^T @ \mathbf{a}_2 / (\eta_1^T @ \eta_1) = \mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_2$$

- Once $\mathbf{q}_1$ is normalized, η_2 can be written as $\eta_2 = (I - \mathbf{q}_1 @ \mathbf{q}_1^T) @ \mathbf{a}_2$ — projecting $\mathbf{a}_2$ onto the orthogonal complement of $\text{span}(\mathbf{q}_1)$
- Similarly, $\eta_3 = (I - \mathbf{q}_1 @ \mathbf{q}_1^T - \mathbf{q}_2 @ \mathbf{q}_2^T) @ \mathbf{a}_3$ — projecting $\mathbf{a}_3$ onto the orthogonal complement of $\text{span}(\mathbf{q}_1, \mathbf{q}_2)$

A WORKED EXAMPLE

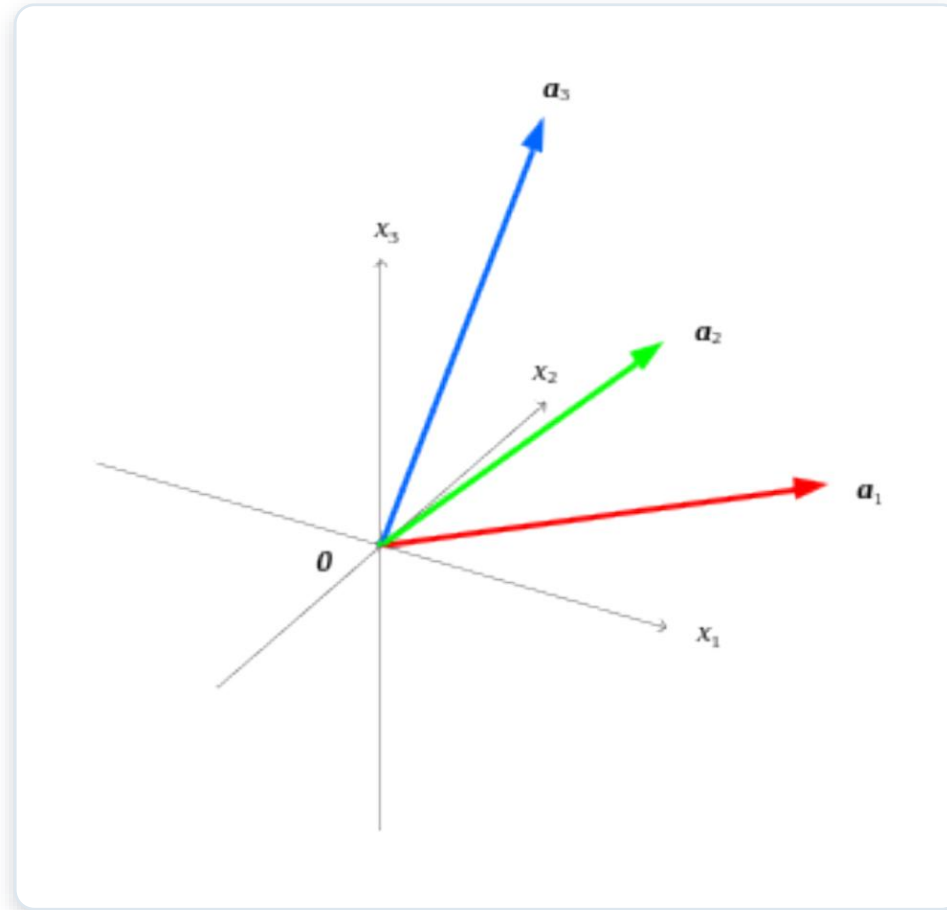
Let's work through Gram–Schmidt orthogonalization step by step on this matrix's columns

$$A = [2,2,0; 2,0,2; 0,2,2] \quad \mathbf{a}_1 = [2,2,0]^T \quad \mathbf{a}_2 = [2,0,2]^T \quad \mathbf{a}_3 = [0,2,2]^T$$

- $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$ are none of them unit vectors and none pairwise orthogonal; but they are linearly independent, so they form a non-orthogonal basis spanning 3D space

Figure 3 · The Positions of $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$

- Though pairwise non-orthogonal, $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$ still span 3D space



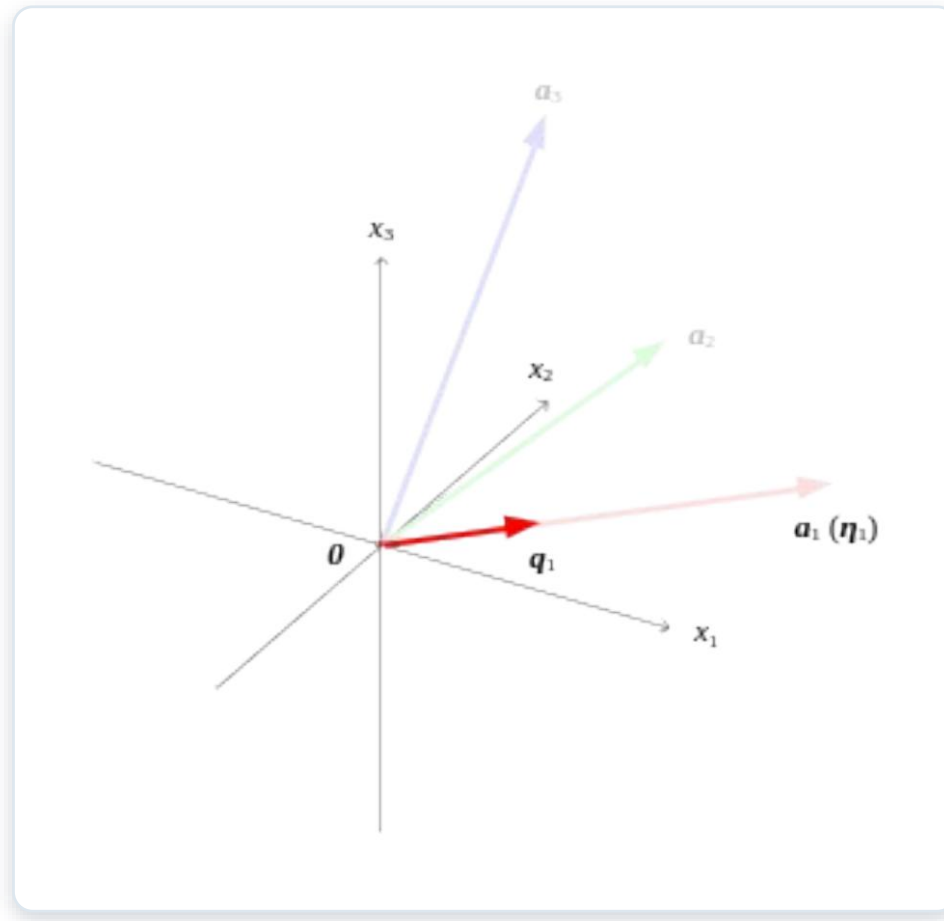
The first step is simplest: $\eta_1 = a_1$, then normalize to get q_1

$$q_1 = \eta_1 / \|\eta_1\| = a_1 / \|a_1\| = [\sqrt{2}/2, \sqrt{2}/2, 0]^T$$

- Clearly q_1 points the same direction as a_1 (η_1)

Figure 4 · Normalizing $\mathbf{a}_1$ (η_1)

- $\mathbf{q}_1$ is just $\mathbf{a}_1$ with the same direction and unit length



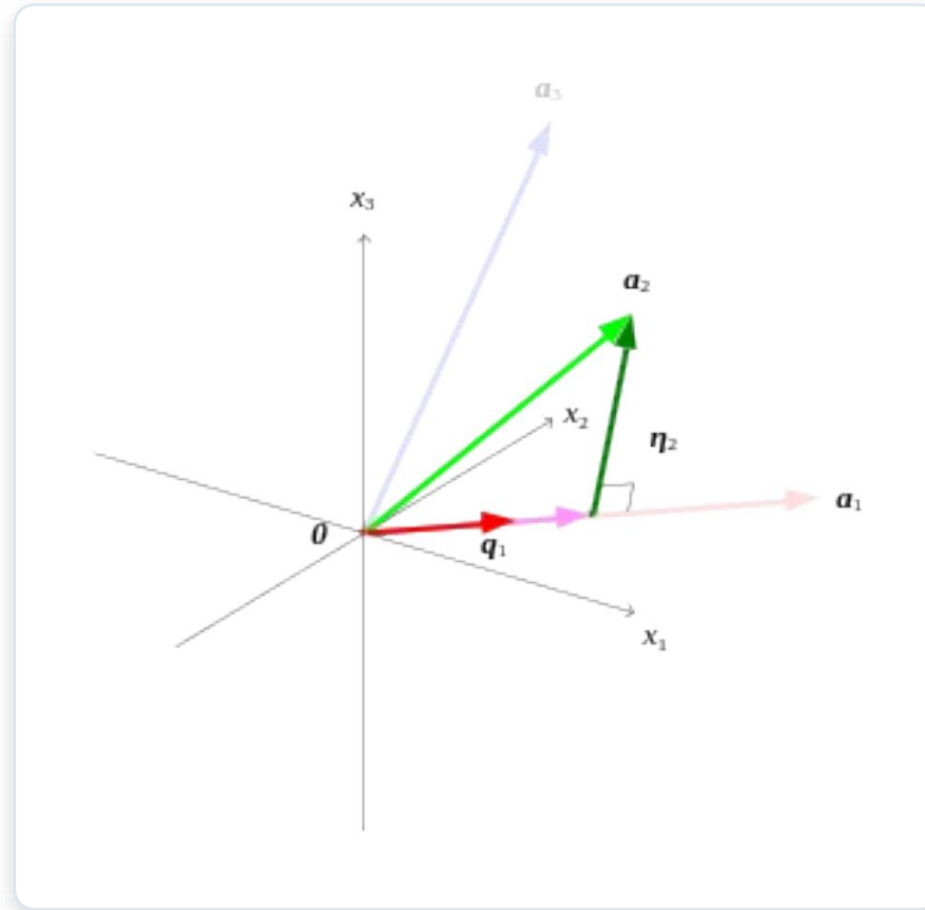
With q_1 in hand, use orthogonal projection to compute η_2

$$\eta_2 = (I - q_1 @ q_1^T) @ a_2 = [1, -1, 2]^T$$

- Build the projection matrix $q_1 @ q_1^T$ first, then subtract it from I to get a matrix that "removes the q_1 -direction component"

Figure 5 · Computing η_2 With an Orthogonal Projection

- η_2 is perpendicular to $q_1 - a_2$ with its q_1 -component removed



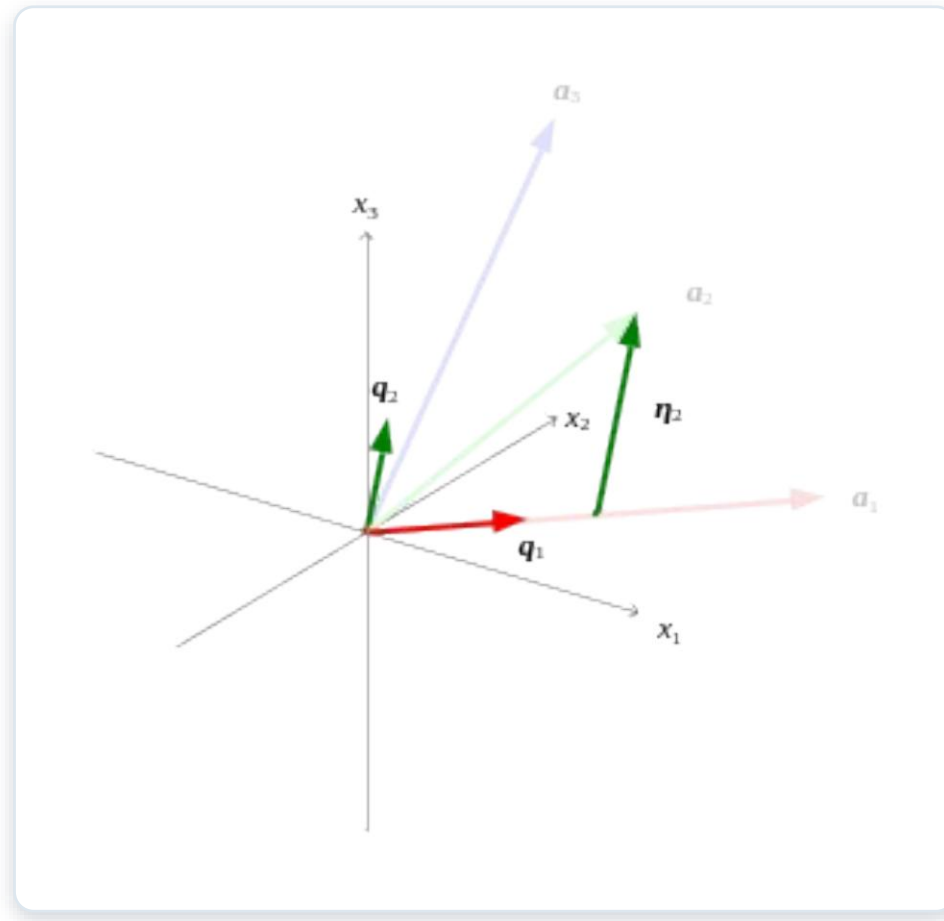
Then, normalizing η_2 gives q_2

$$q_2 = \eta_2 / \|\eta_2\| = [\sqrt{6}/6, -\sqrt{6}/6, \sqrt{6}/3]^T$$

- With q_2 computed, let's do some "extra" analysis to build geometric intuition for this step

Figure 6 · Normalizing η_2

- q_2 has the same direction as η_2 and unit length



This step is equivalent to orthogonally decomposing $\mathbf{a}_2$ into two parts

$$\mathbf{a}_2 = \mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_2 + \boldsymbol{\eta}_2$$

- $\mathbf{a}_2$ decomposes into a part parallel to $\mathbf{q}_1$ ($\mathbf{a}_1, \boldsymbol{\eta}_1$), and a part perpendicular to $\mathbf{q}_1$
- Remove the component of $\mathbf{a}_2$ that's parallel to $\mathbf{a}_1$; the remaining perpendicular part ($\boldsymbol{\eta}_2$) is exactly $\mathbf{a}_2$'s "unique" contribution

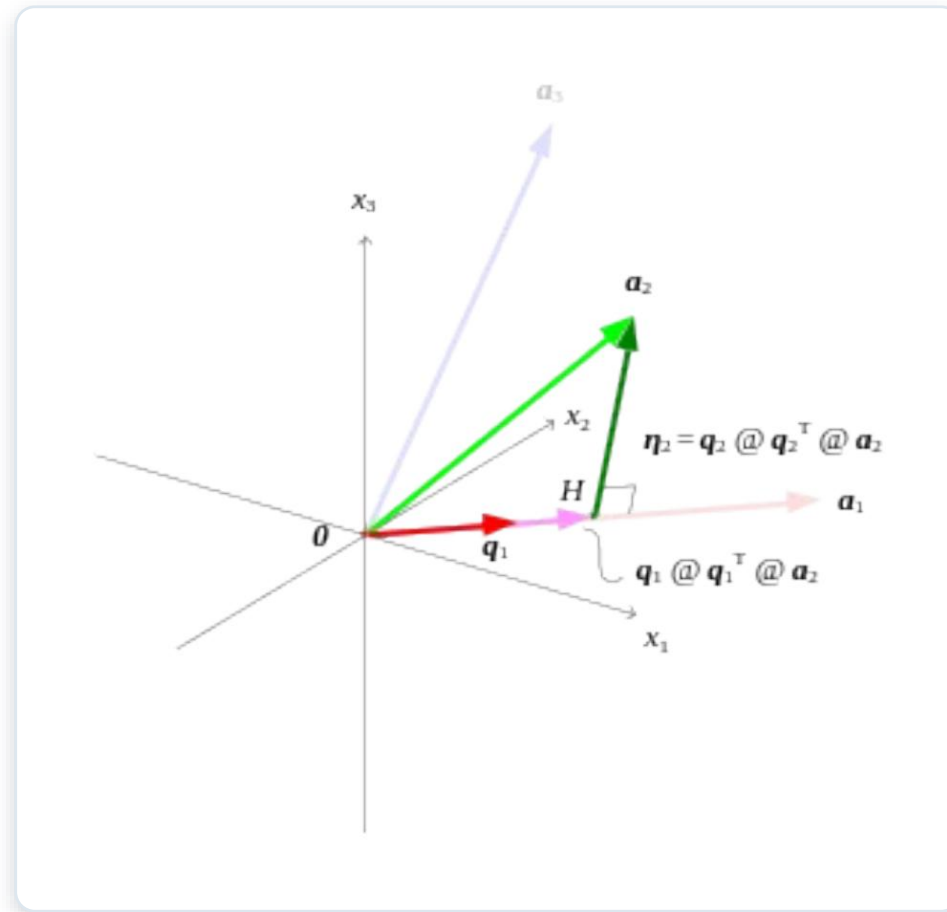
Now that we have $\mathbf{q}_2$, $\boldsymbol{\eta}_2$ can be written as

$$\boldsymbol{\eta}_2 = \mathbf{q}_2 @ \mathbf{q}_2^T @ \mathbf{a}_2 \quad \mathbf{a}_2 = \mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_2 + \mathbf{q}_2 @ \mathbf{q}_2^T @ \mathbf{a}_2 = (\mathbf{q}_1 @ \mathbf{q}_1^T + \mathbf{q}_2 @ \mathbf{q}_2^T) @ \mathbf{a}_2$$

- Note: $\mathbf{q}_1 @ \mathbf{q}_1^T + \mathbf{q}_2 @ \mathbf{q}_2^T$ is not full rank, so this "square matrix" has no inverse

Figure 7 · Orthogonally Decomposing $\mathbf{a}_2$

- Point H is $\mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_2$; the leftover $\boldsymbol{\eta}_2$ is perpendicular to it



In this expression, first take the scalar projection, then multiply by $\mathbf{q}_1$ for the vector projection

$$\mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_2 = [1, 1, 0]^T = \mathbf{a}_1/2$$

- $\mathbf{q}_1^T @ \mathbf{a}_2$ is the scalar projection — $\mathbf{a}_2$'s coordinate along $\mathbf{q}_1$; multiplying by $\mathbf{q}_1$ gives the vector projection — this is the "two-step projection" from earlier
- Clearly $\mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_2$ is parallel to $\mathbf{a}_1$; you can verify $\mathbf{n}_2$ is orthogonal to it, and that the two sum back to $\mathbf{a}_2$

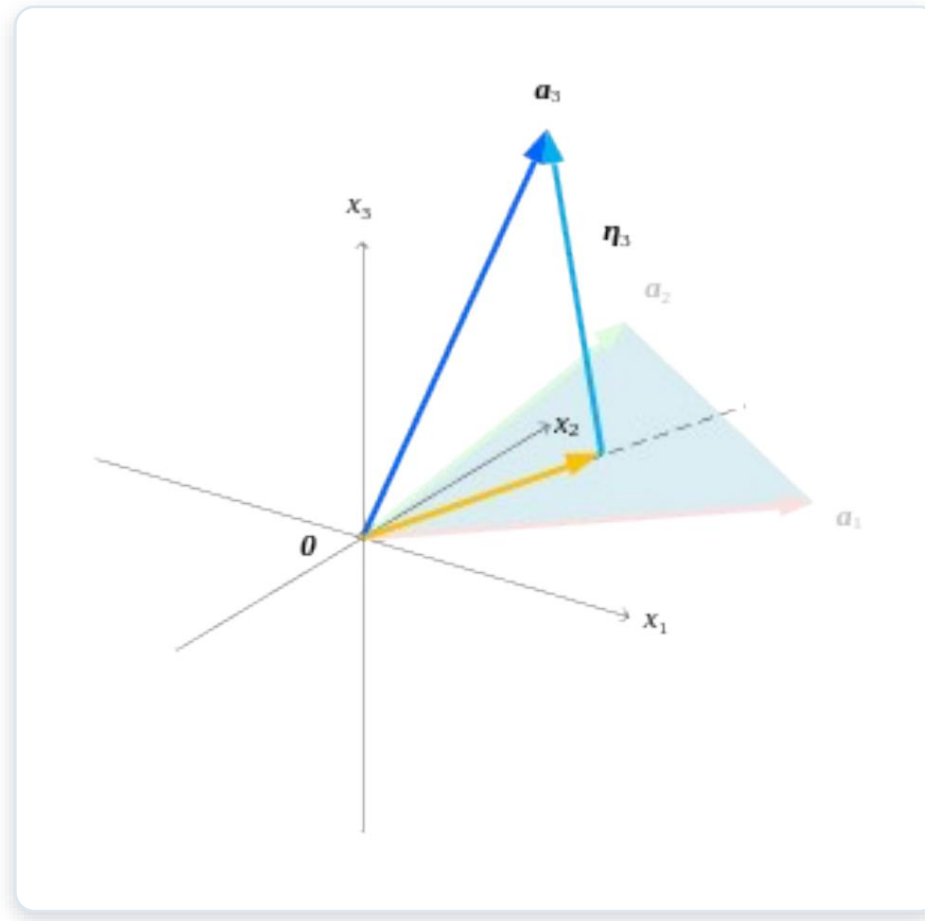
With $\mathbf{q}_1$ and $\mathbf{q}_2$ in hand, use orthogonal projection to compute $\boldsymbol{\eta}_3$

$$\boldsymbol{\eta}_3 = (\mathbf{I} - \mathbf{q}_1 @ \mathbf{q}_1^T - \mathbf{q}_2 @ \mathbf{q}_2^T) @ \mathbf{a}_3 = [-4/3, 4/3, 4/3]^T$$

- Geometrically, this step projects $\mathbf{a}_3$ onto the plane spanned by $[\boldsymbol{\eta}_1, \boldsymbol{\eta}_2]$; $\boldsymbol{\eta}_3$ is the part of $\mathbf{a}_3$ not on that plane
- $\text{span}(\boldsymbol{\eta}_1, \boldsymbol{\eta}_2)$ is equivalent to $\text{span}(\mathbf{q}_1, \mathbf{q}_2)$: $\boldsymbol{\eta}_3$ is perpendicular to both $\boldsymbol{\eta}_1$ and $\boldsymbol{\eta}_2$, so $\text{span}(\boldsymbol{\eta}_3)$ and $\text{span}(\boldsymbol{\eta}_1, \boldsymbol{\eta}_2)$ are orthogonal complements

Figure 8 · Computing η_3 With an Orthogonal Projection

- Remove $\mathbf{a}_3$'s parts along $\mathbf{a}_1, \mathbf{a}_2$; η_3 is what remains — its "unique" part



Then, normalizing η_3 gives q_3

$$q_3 = \eta_3 / \|\eta_3\| = [-\sqrt{3}/3, \sqrt{3}/3, \sqrt{3}/3]^T$$

- We've now finished orthogonalizing $[a_1, a_2, a_3]$ into $[q_1, q_2, q_3]$

Figure 9 · Normalizing η_3

- q_3 has the same direction as η_3 and unit length

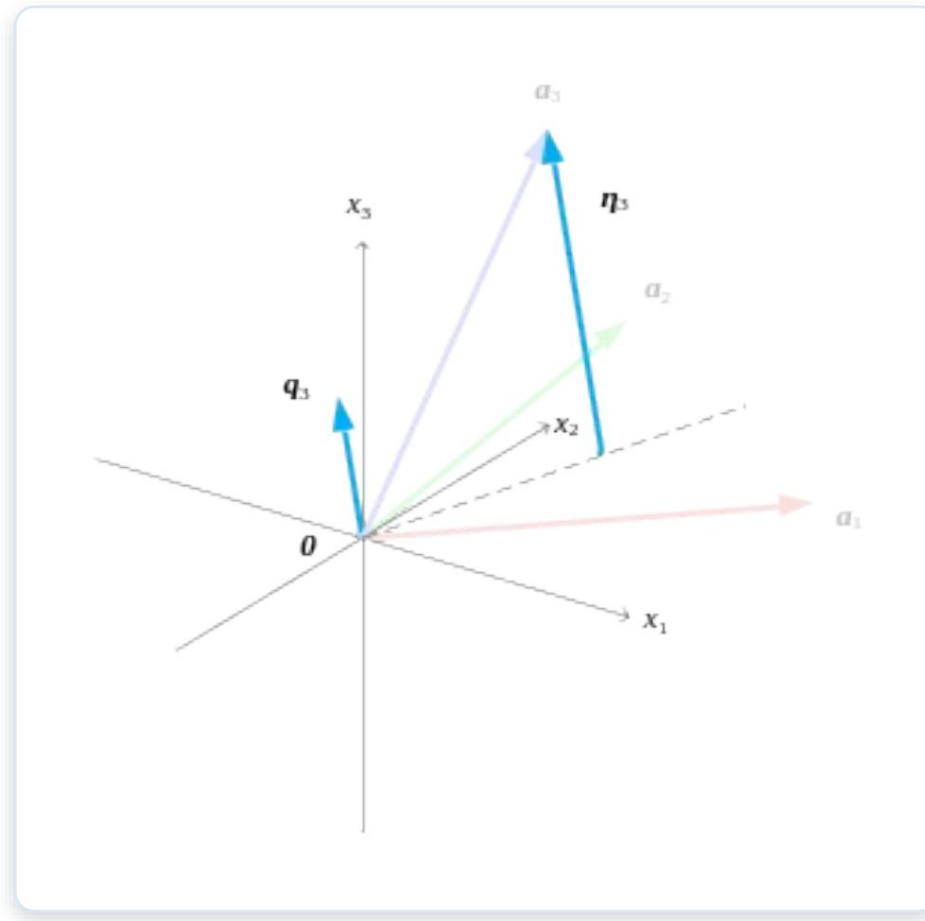
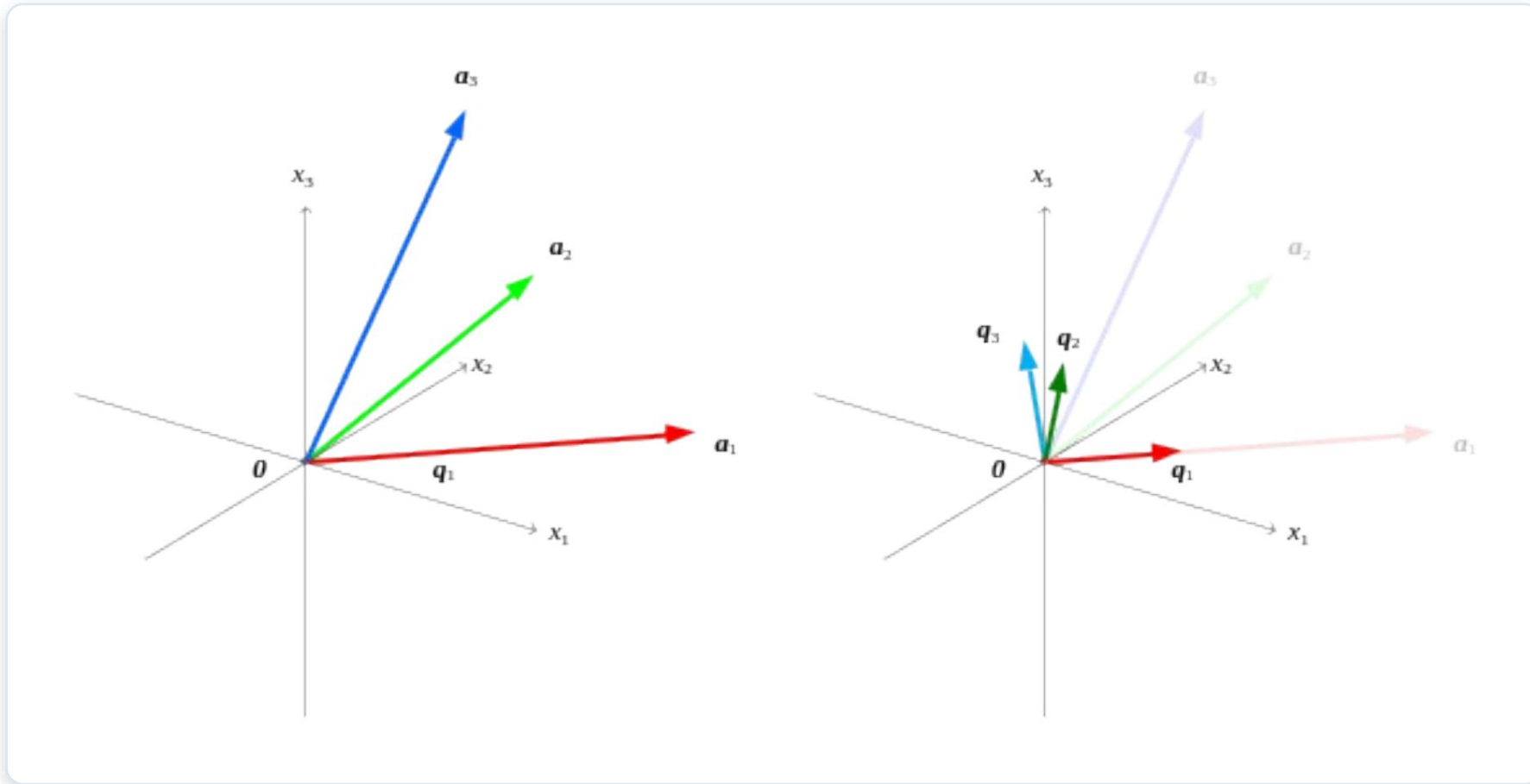


Figure 10 · Orthogonalizing $[a_1, a_2, a_3]$ Into $[q_1, q_2, q_3]$

- Left: non-orthogonal basis $[a_1, a_2, a_3]$; right: orthonormal basis $[q_1, q_2, q_3]$



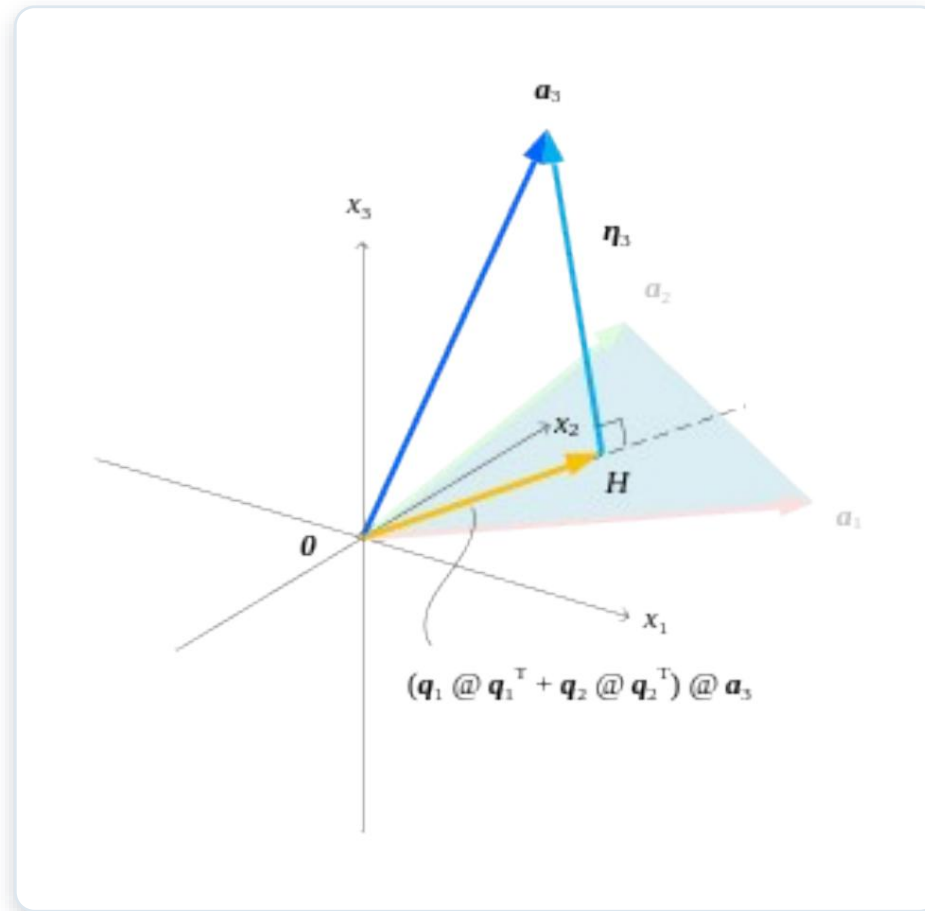
But we're not done yet — $\mathbf{a}_3$ can also be orthogonally decomposed into two parts

$$\mathbf{a}_3 = (\mathbf{q}_1 @ \mathbf{q}_1^T + \mathbf{q}_2 @ \mathbf{q}_2^T) @ \mathbf{a}_3 + \boldsymbol{\eta}_3$$

- $\mathbf{a}_3$ decomposes into a part on the plane $\text{span}(\mathbf{q}_1, \mathbf{q}_2)$, and a part perpendicular to $\text{span}(\mathbf{q}_1, \mathbf{q}_2)$
- Remove the on-plane component; the remaining perpendicular part ($\boldsymbol{\eta}_3$) is $\mathbf{a}_3$'s "unique" contribution

Figure 11 · Orthogonally Decomposing $\mathbf{a}_3$

- Point H is where $(\mathbf{q}_1 @ \mathbf{q}_1^T + \mathbf{q}_2 @ \mathbf{q}_2^T) @ \mathbf{a}_3$ lands on the plane $\text{span}(\mathbf{q}_1, \mathbf{q}_2)$



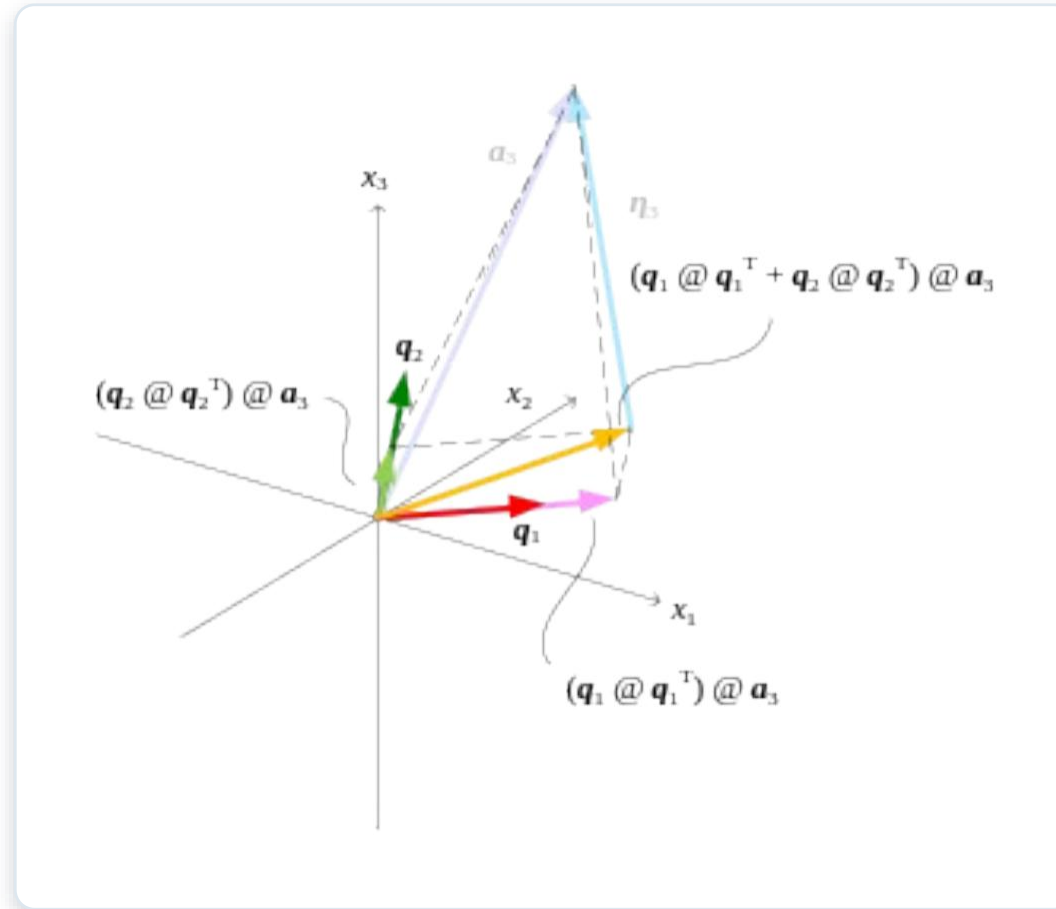
This on-plane component can be orthogonally decomposed further

$$\mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_3 = [1, 1, 0]^T$$

- Geometrically, $\mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_3$ is $\mathbf{a}_3$'s orthogonal projection onto direction $\mathbf{q}_1$

Figure 12 · Decomposing $(\mathbf{q}_1 @ \mathbf{q}_1^T + \mathbf{q}_2 @ \mathbf{q}_2^T) @ \mathbf{a}_3$

- The on-plane component splits further into projections along $\mathbf{q}_1$ and along $\mathbf{q}_2$



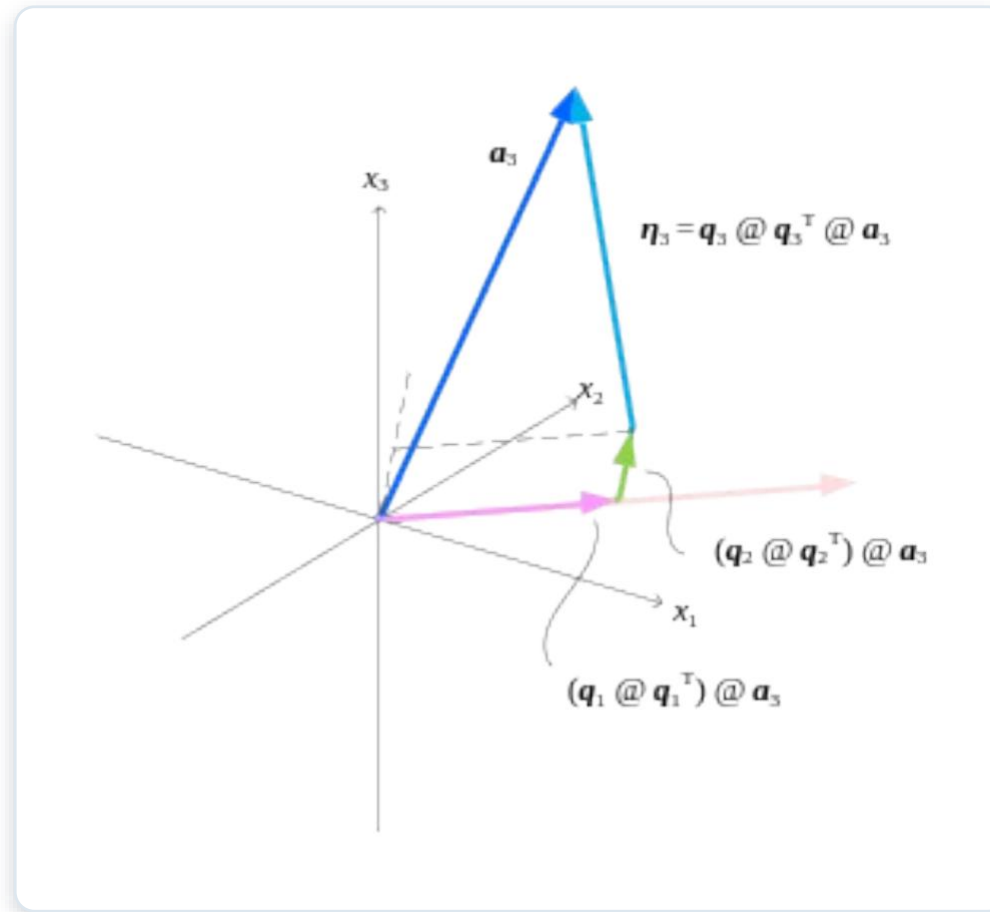
Three parts sum head to tail, restoring $\mathbf{a}_3$ exactly

$$\mathbf{q}_2 @ \mathbf{q}_2^T @ \mathbf{a}_3 = [1/3, -1/3, 2/3]^T \quad \boldsymbol{\eta}_3 + \mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_3 + \mathbf{q}_2 @ \mathbf{q}_2^T @ \mathbf{a}_3 = \mathbf{a}_3$$

- Geometrically, $\mathbf{q}_2 @ \mathbf{q}_2^T @ \mathbf{a}_3$ is $\mathbf{a}_3$'s orthogonal projection onto direction $\mathbf{q}_2$
- Try verifying yourself that $\boldsymbol{\eta}_3$, $\mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_3$, and $\mathbf{q}_2 @ \mathbf{q}_2^T @ \mathbf{a}_3$ are pairwise orthogonal

Figure 13 · Three Vectors Restoring $\mathbf{a}_3$ Head to Tail

- η_3 , $\mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_3$, and $\mathbf{q}_2 @ \mathbf{q}_2^T @ \mathbf{a}_3$ sum head to tail to restore $\mathbf{a}_3$ exactly



With $\mathbf{q}_3$ in hand, $\boldsymbol{\eta}_3$ can be rewritten using $\mathbf{q}_3$, and this gives us

$$\mathbf{a}_3 = (\mathbf{q}_1 @ \mathbf{q}_1^T + \mathbf{q}_2 @ \mathbf{q}_2^T + \mathbf{q}_3 @ \mathbf{q}_3^T) @ \mathbf{a}_3 \quad \mathbf{q}_1 @ \mathbf{q}_1^T + \mathbf{q}_2 @ \mathbf{q}_2^T + \mathbf{q}_3 @ \mathbf{q}_3^T = \mathbf{I}$$

- Here $\mathbf{a}_1 = (\mathbf{q}_1 @ \mathbf{q}_1^T) @ \mathbf{a}_1$ — this actually leads into eigenvalue decomposition $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$: for a square matrix $\mathbf{A}$, finding nonzero vectors $\mathbf{v}$ and scalars λ such that $\mathbf{A}$ only stretches, never redirects, along these directions
- Eigenvalue decomposition is a major topic covered later in this book

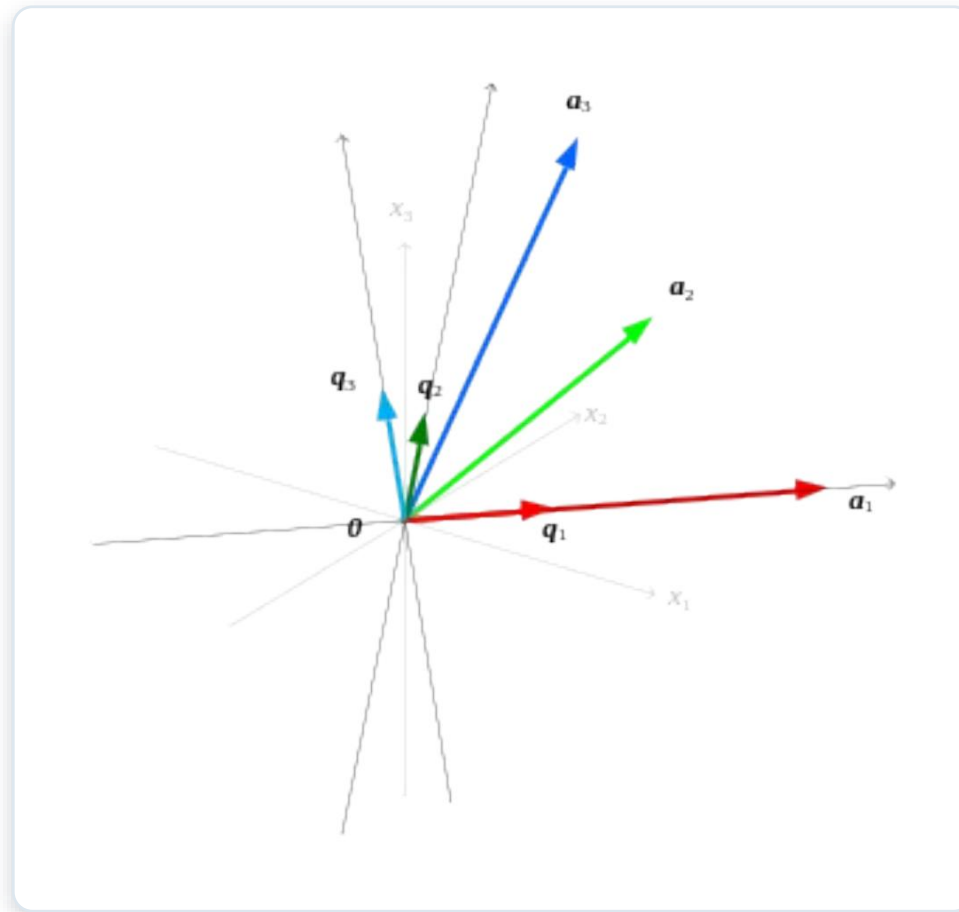
a_1, a_2, a_3 naturally have their own coordinates in $[q_1, q_2, q_3]$ too

$$\mathbf{a}_1 = [2, 2, 0]^T \quad \mathbf{a}_2 = [2, 0, 2]^T \quad \mathbf{a}_3 = [0, 2, 2]^T \quad (\text{in the } [\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3] \text{ coordinate system})$$

- $[q_1, q_2, q_3]$ is an orthonormal basis; the parallelepiped it spans is a rotated unit cube

Figure 14 · $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$ in Orthonormal Basis $[\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3]$

- Let's compute each of these coordinates one by one



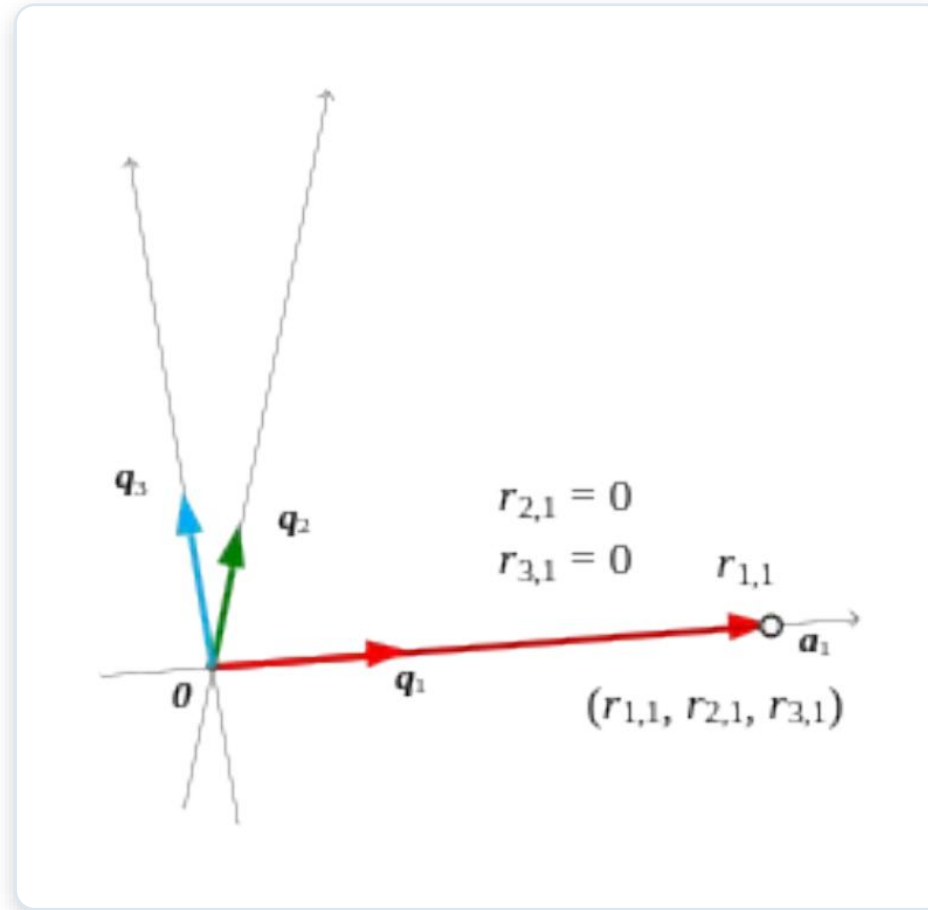
Let $\mathbf{a}_1$'s coordinates in $[\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3]$ be $(r_{1,1}, r_{2,1}, r_{3,1})$

$$\mathbf{a}_1 = r_{1,1}\mathbf{q}_1 + r_{2,1}\mathbf{q}_2 + r_{3,1}\mathbf{q}_3 \quad r_{1,1} = \mathbf{a}_1 \cdot \mathbf{q}_1 = 2\sqrt{2} \quad r_{2,1} = r_{3,1} = 0$$

- Vector $\mathbf{a}_1$'s coordinate along direction $\mathbf{q}_1$ is their inner product — the scalar projection; since $\mathbf{a}_1$ is parallel to $\mathbf{q}_1$, its scalar projections onto $\mathbf{q}_2$ and $\mathbf{q}_3$ are 0

Figure 15 · a_1 's Coordinates in $[q_1, q_2, q_3]$

- a_1 's coordinates in the new basis are $(2\sqrt{2}, 0, 0)$ — only a component along q_1



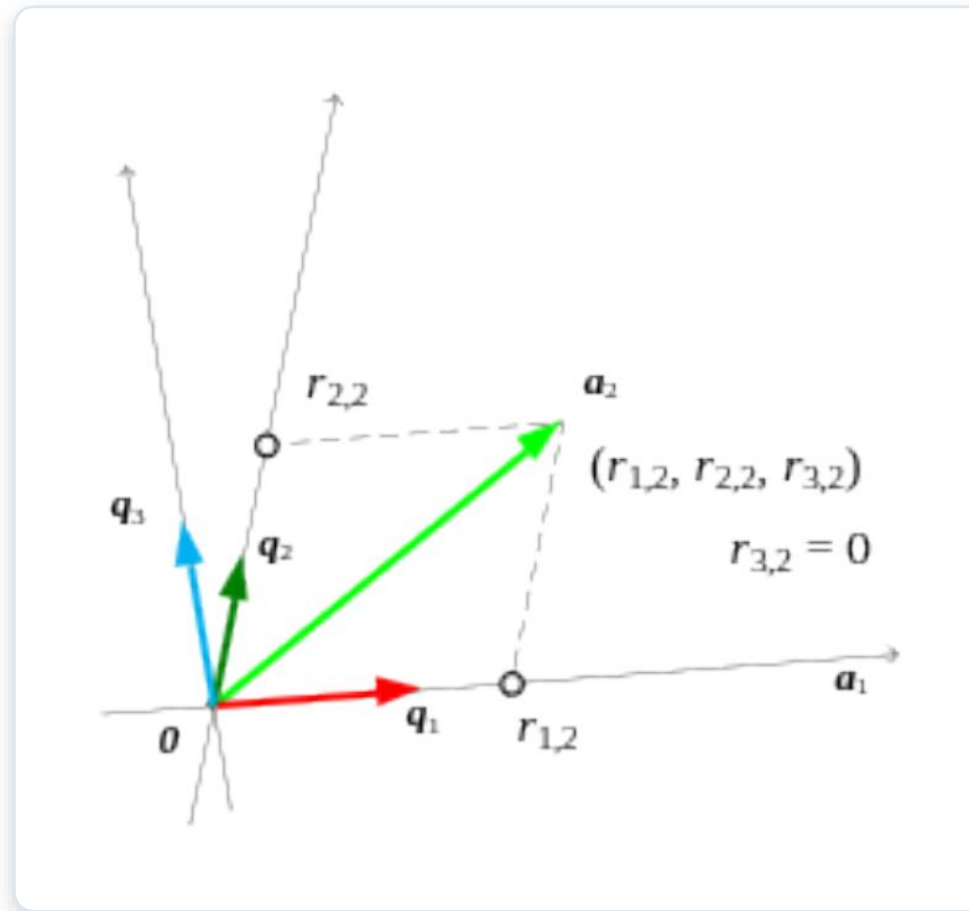
Let $\mathbf{a}_2$'s coordinates in $[\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3]$ be $(r_{1,2}, r_{2,2}, r_{3,2})$

$$r_{1,2} = \mathbf{a}_2 \cdot \mathbf{q}_1 = \sqrt{2} \quad r_{2,2} = \mathbf{a}_2 \cdot \mathbf{q}_2 = \sqrt{6} \quad r_{3,2} = 0$$

- $\mathbf{a}_2$ lies in $\text{span}(\mathbf{q}_1, \mathbf{q}_2)$, so $\mathbf{a}_2$'s scalar projection onto $\mathbf{q}_3$ is 0

Figure 16 · a_2 's Coordinates in $[q_1, q_2, q_3]$

- a_2 's coordinates in the new basis are $(\sqrt{2}, \sqrt{6}, 0)$ — lying on the plane spanned by q_1 and q_2



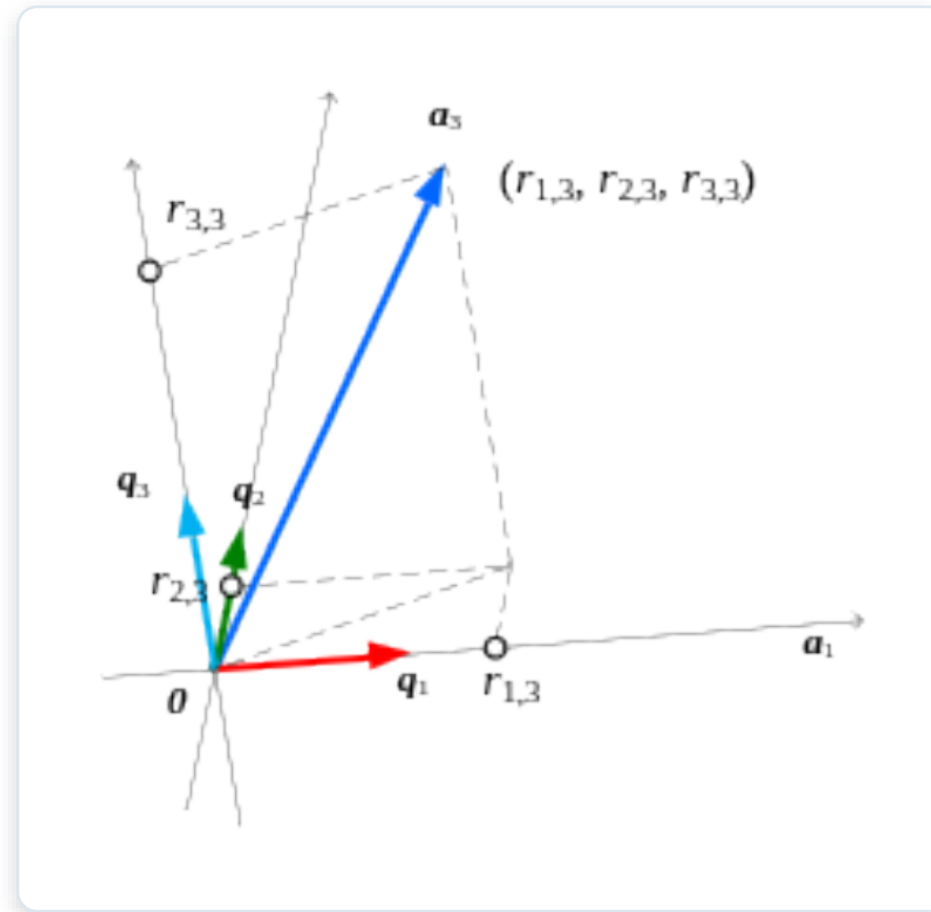
Let $\mathbf{a}_3$'s coordinates in $[\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3]$ be $(r_{1,3}, r_{2,3}, r_{3,3})$

$$r_{1,3} = \mathbf{a}_3 \cdot \mathbf{q}_1 = \sqrt{2} \quad r_{2,3} = \mathbf{a}_3 \cdot \mathbf{q}_2 = \sqrt{6}/3 \quad r_{3,3} = \mathbf{a}_3 \cdot \mathbf{q}_3 = 4\sqrt{3}/3$$

- This time all three coordinates are nonzero, since $\mathbf{a}_3$ has a component along each of $\mathbf{q}_1$, $\mathbf{q}_2$, and $\mathbf{q}_3$

Figure 17 · $\mathbf{a}_3$'s Coordinates in $[\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3]$

- $\mathbf{a}_3$'s coordinates in the new basis are $(\sqrt{2}, \sqrt{6}/3, 4\sqrt{3}/3)$



This gives us a complete set of vector decompositions

$$\begin{aligned} \mathbf{a}_1 &= \mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_1 & \mathbf{a}_2 &= \mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_2 + \mathbf{q}_2 @ \mathbf{q}_2^T @ \mathbf{a}_2 & \mathbf{a}_3 &= \\ & \mathbf{q}_1 @ \mathbf{q}_1^T @ \mathbf{a}_3 + \mathbf{q}_2 @ \mathbf{q}_2^T @ \mathbf{a}_3 + \mathbf{q}_3 @ \mathbf{q}_3^T @ \mathbf{a}_3 \end{aligned}$$

- Once again we see orthogonal projection and vector decomposition at work

Arranging the three steps in order gives $A = Q @ R$

$$[a_1, a_2, a_3] = [q_1, q_2, q_3] @ R \quad Q = [\sqrt{2}/2, \sqrt{6}/6, -\sqrt{3}/3; \sqrt{2}/2, -\sqrt{6}/6, \sqrt{3}/3; 0, \sqrt{6}/3, \sqrt{3}/3]$$

- R is upper triangular; in this example Q is orthogonal, since its columns are unit vectors and pairwise orthogonal

Using this section's orthogonal-projection relations, upper-triangular R can also be written as

$$R = [q_1^T @ a_1, q_1^T @ a_2, q_1^T @ a_3; 0, q_2^T @ a_2, q_2^T @ a_3; 0, 0, q_3^T @ a_3] = [2\sqrt{2}, \sqrt{2}, \sqrt{2}; 0, \sqrt{6}, \sqrt{6}/3; 0, 0, 4\sqrt{3}/3]$$

- R 's columns r_1, r_2, r_3 are exactly A 's columns a_1, a_2, a_3 expressed as coordinates in $[q_1, q_2, q_3]$

$$A = QR$$

Finally, put the three pieces together as $A = QR$

$$A = QR$$

- This is exactly next section's QR decomposition — at its core, Gram–Schmidt orthogonalization is the process of turning A 's columns into Q 's columns

What Does Gram–Schmidt Orthogonalization Reveal?

The orthogonalization process: project step by step, subtract, normalize — each step yields a new direction orthogonal to the ones already found

A special feature of orthonormal basis $[q_1, q_2, \dots, q_p]$: q_1 is always parallel to a_1

Every vector can be orthogonally decomposed into a "known-direction part" plus a "unique-direction part"

$A = QR$: Q 's columns form the orthonormal basis, R 's columns are the original vectors' coordinates in that basis

Glossary

Gram–Schmidt Orthogonalization 格拉姆-施密特正交化—— builds an orthonormal basis via step-by-step projection and normalization

Orthogonal Projection 正交投影—— $\mathbf{q} @ \mathbf{q}^T @ \mathbf{a}$, the projection of vector $\mathbf{a}$ onto direction $\mathbf{q}$

Unique Direction 独特方向—— the orthogonal part of a vector left over after removing its known-direction components

Upper Triangular Matrix 上三角矩阵—— a square matrix with all entries below the diagonal equal to 0, such as this section's R

QR Decomposition QR分解—— $A = Q @ R$, with Q orthogonal and R upper triangular

Orthogonalization 正交化—— turning a linearly independent set into a pairwise-orthogonal set

Orthogonal Complement 正交补—— the space of all vectors perpendicular to every vector in a given subspace

Semi-Orthogonal Matrix 半正交矩阵—— the matrix Q obtained by orthogonalizing a full-column-rank matrix, with orthonormal columns

Scalar Projection 标量投影—— vector $\mathbf{a}$'s coordinate along unit vector $\mathbf{q}$, equal to their inner product

Eigenvalue Decomposition 特征值分解—— $A @ \mathbf{v} = \lambda \mathbf{v}$, finding the directions along which square matrix A only stretches, never redirects

EXPLORE FURTHER

Further Reading

Recap · 9.3 Properties of Orthogonal Matrices

Previous section — the Gram matrix, determinant, pseudo-inverse, and other algebraic properties

Further Reading · 9.5 QR Decomposition

Next section in this chapter — decomposing a matrix into an orthogonal matrix and an upper-triangular matrix

Further Reading · Eigenvalue Decomposition

Later in this book — the geometric and algebraic meaning behind $A\mathbf{v}=\lambda\mathbf{v}$

Open-Source Repository

All code and figures for the book: <https://github.com/Visualize-ML>

The Columns of the Original A, and of Q1's Column-Swapped Version

Vector Set	Column 1	Column 2	Column 3
Original [a1,a2,a3]	[2,2,0] ^T	[2,0,2] ^T	[0,2,2] ^T
Q1 swap [a1',a2',a3']	[2,0,2] ^T	[2,2,0] ^T	[0,2,2] ^T

Q1 – Q9

Q1 For $[a1', a2', a3']$ (the column-swapped version in the table above), work through Gram–Schmidt orthogonalization by hand to find $q1'$, $q2'$, $q3'$.

Q3 Compute the determinant of the original matrix $A=[a1, a2, a3]$, and verify A has full column rank (linearly independent columns).

Q5 For $Q1$'s $[a1', a2', a3']$, compute $\eta2'=(I-q1'@q1'^T)@a2'$.

Q7 For $Q1$'s $[a1', a2', a3']$, compute $\eta3'=(I-q1'@q1'^T-q2'@q2'^T)@a3'$, and normalize to get $q3'$.

Q9 Compute $Q1$'s upper-triangular R' , and verify $A'=Q'@R'$ (A' being the column-swapped matrix).

Q2 Compute $\|a1\|$, $\|a2\|$, $\|a3\|$ for the original $a1$, $a2$, $a3$, and verify they are pairwise non-orthogonal (nonzero inner products).

Q4 For $Q1$'s swapped $a1'$, compute $q1'=a1'/\|a1'\|$, and compare with the original $q1$.

Q6 Continuing from the previous question, normalize $\eta2'$ to get $q2'$.

Q8 Using the $Q'=[q1', q2', q3']$ found in $Q1$, verify $Q'^T@Q'=I$ — confirming Q' is indeed orthogonal.

Q10 – Q17

Q10 Explain why q_1' changes when A 's column order is swapped — q_1 is always parallel to whichever column is orthogonalized first.

Q12 Use `numpy.linalg.qr(A)` to decompose this section's matrix A , and compare the resulting Q against your hand calculation, noting any sign differences.

Q14 Write a Python function `is_orthonormal(Q)` that checks whether a set of column vectors is pairwise orthogonal and unit length.

Q16 Use matplotlib 3D plotting to visualize a_1, a_2, a_3 alongside the orthogonalized q_1, q_2, q_3 .

Q11 Write a Python function `gram_schmidt(A)` implementing Gram–Schmidt orthogonalization from scratch (no library calls), and test it on this section's matrix A .

Q13 Explore: why might numpy's QR result differ from the hand-computed Q by a sign? How does this relate to the uniqueness of QR decomposition?

Q15 Extend this section's $[a_1, a_2, a_3]$ to any 3 of 4 linearly independent 3D vectors, and redo Gram–Schmidt orthogonalization.

Q17 Explore: change one entry of matrix A and observe how q_1, q_2, q_3 change in response.

Q18 – Q25

Q18 Verify projection matrix $q_1 q_1^T$ is idempotent (equal to itself when multiplied by itself) and symmetric.

Q20 Compute the determinant of this section's R , and compare it with $\det(A)$ and $\det(Q)$ to see how they relate.

Q22 Explore: classical Gram–Schmidt can be numerically unstable — look up "modified Gram–Schmidt" and compare the two approaches.

Q24 Write a Python function `project_onto_basis(a, Q)` that computes a vector a 's coordinates in an existing orthonormal basis Q (i.e., the corresponding column of R).

Q19 Verify $q_1 q_1^T + q_2 q_2^T + q_3 q_3^T = I$, and explain the geometric meaning of this identity.

Q21 Verify R is indeed upper triangular: explain why its strictly-lower-triangular entries must be 0.

Q23 Compute $\det(Q)$, and explain what its sign (+1 or -1) means for a right-handed versus left-handed coordinate system.

Q25 Preview thought: the next section systematically covers QR decomposition — how does $A = Q @ R$ help when solving a linear system $Ax = b$?

10 Prompts to Explore With AI

- 1 Explain why every step of Gram–Schmidt orthogonalization needs to "subtract the projection onto directions already found".
- 2 Explain why q_1 is always parallel to a_1 in the orthonormal basis $[q_1, q_2, \dots, q_p]$.
- 3 Explain what "orthogonal decomposition" means: how does a vector split into a "known-direction part" and a "unique-direction part"?
- 4 Use Python to demonstrate the full Gram–Schmidt computation, and compare the result against `numpy.linalg.qr`.
- 5 Explain why $q_1 q_1^T + q_2 q_2^T + q_3 q_3^T$ equals the identity matrix I .

10 Prompts to Explore With AI

6 Explain the geometric meaning of each column of R : why are they the original vectors' coordinates in the new basis?

7 Explain why R must be upper triangular, and how this relates to the order in which Gram–Schmidt orthogonalization proceeds.

8 Explain the possible connection between Gram–Schmidt orthogonalization and eigenvalue decomposition $A\mathbf{v}=\lambda\mathbf{v}$.

9 Explain the difference in numerical stability between classical Gram–Schmidt and modified Gram–Schmidt.

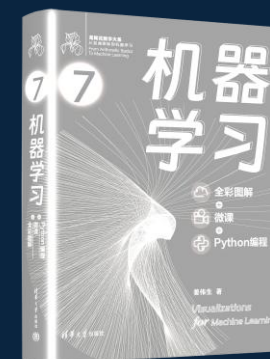
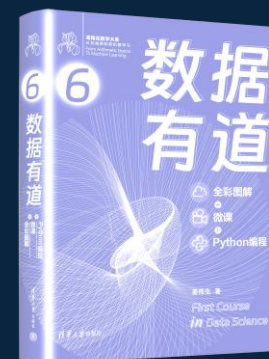
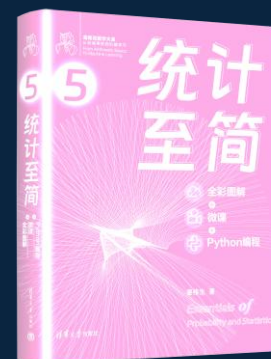
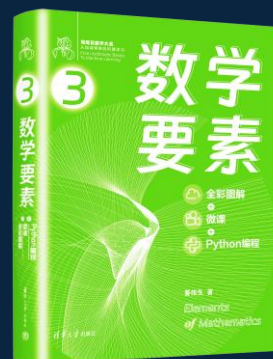
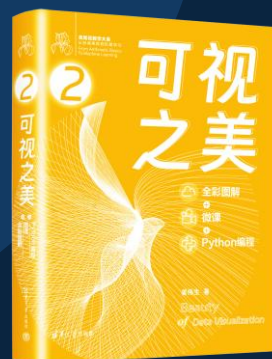
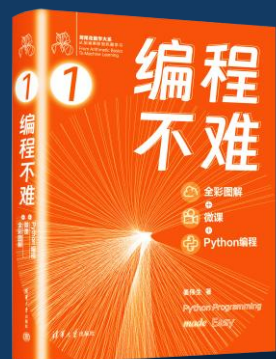
10 Explain how the result $A=Q\mathbf{R}$ sets up the next section's QR decomposition.

数学

数学基础

线性代数

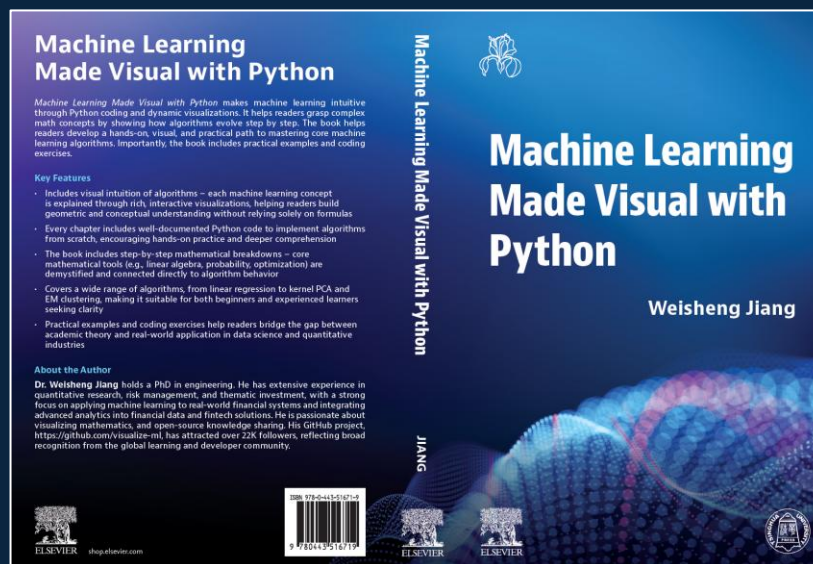
概率统计



Python编程

数据可视化

工具



数据分析
图和网络

回归、降维
分类、聚类

实践